

## AI Question 01:

### **Exercise 1: Robot Navigation in a Grid**

#### **Problem Formulation**

- **Description:** A robot is placed in a 5x5 grid. The robot needs to navigate from the start position (0, 0) to the goal position (4, 4). There are obstacles in some cells of the grid that the robot cannot pass through. Diagonal nodes are not allowed.

#### **Question**

Formulate this problem as an AI search problem.

#### **Solution:**

- **State:** The position of the robot in the grid, represented as (x, y).
- **Initial State:** (0, 0)
- **Goal State:** (4, 4)
- **Actions:** Move Up, Move Down, Move Left, Move Right.
- **Transition Model:** Given a state (x, y) and an action, return the new state if it is within the grid bounds and not an obstacle. The next cell must be an adjacent cell. One of the values of x and y can change in one step, not both, since diagonal moves are not allowed.
- **Step Cost:** Each move has a cost of 1

### **Exercise 2: Sudoku Solver**

Fill a 9x9 Sudoku grid such that each row, column, and 3x3 sub-grid contains the digits 1 to 9.

#### **Question**

Formulate this problem as an AI search problem.

#### **Solution**

#### **Problem Formulation**

- **State:** Any 9x9 grid, each cell can be empty or have a value ranging from 1 to 9.
- **Initial State:** Any grid configuration with cell values between 1 and 9.
- **Goal State:** A fully filled grid that satisfies the Sudoku constraints.
- **Actions:** Change the value of any cell with a value between 1 and 9 without violating the rules, the same value cannot occur twice in the same row, column and 3x3 sub-grids.
- **Transition Model:** a change in any cell value will lead to a new state.
- **Step Cost:** Each step cost has a value of 1

### Exercise 3: Missionaries and Cannibals

Three missionaries and three cannibals are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side without ever leaving a group of missionaries outnumbered by cannibals in any place.

#### Question

Formulate this problem as an AI search problem.

#### Solution:

##### Problem Formulation

Each state is represented by the following tuple  $(M_{\text{left}}, C_{\text{left}}, B)$ , where:

$M_{\text{left}}$  is the number of missionaries on the left bank.

$C_{\text{left}}$  is the number of cannibals on the left bank.

$B$  indicates the position of the boat (1 for left bank, 0 for right bank).

- **State:** The number of missionaries and cannibals on each side of the river and the position of the boat.
- **Initial State:** All missionaries and cannibals are on the starting side (3, 3, 1).
- **Goal State:** All missionaries and cannibals are on the opposite side (0, 0, 0).
- **Actions:** Move one or two people from one side of the river to the other using the boat.
- **Transition Model:** Any change of the number of missionaries and cannibals on any side of the rivers will lead to a new state (for sure the change of the boat location is considered as a new state, but the boat will not change location alone).

## Ai Question 02:

### **Question 1**

---

A smart thermostat agent controls the temperature of a home. The agent's task is to monitor the changes of temperature, humidity outside and inside the house, and control the air conditioners distributed within the house to maintain a calm temperature inside the house.

- a) Give the PEAS description of the above agent.

Performance:

Environment:

Actuators:

Sensors:

- b) Determine the following environment properties of the speed monitor agent

| Observability ? | Deterministic /<br>Stochastic ? | Static / Dynamic? | Single / Multi<br>agent? |
|-----------------|---------------------------------|-------------------|--------------------------|
|-----------------|---------------------------------|-------------------|--------------------------|

### **Solution:**

Performance: Energy efficiency, maintaining desired temperature, user comfort, response time.

Environment: Home interior, outdoor weather conditions.

Actuators: Heating system, cooling system, fans.

Sensors: Temperature sensors, humidity sensors, occupancy sensors.

| Observability ?  | Deterministic /<br>Stochastic ? | Static / Dynamic? | Single / Multi<br>agent? |
|------------------|---------------------------------|-------------------|--------------------------|
| Fully Observable | Deterministic                   | Static            | Single Agent             |

## Question 2

---

### Problem:

The Blocks World problem involves a set of blocks, each represented by a letter and a flat table surface. The problem's rules are relatively simple:

- Blocks can be stacked on top of each other.
- Only one block can be placed directly on top of another block.
- To move a block that has other blocks stacked on top of it, you must first unstack the blocks above it until the desired block is free to move.
- Blocks can also be placed directly on the table.

In this question we are going to consider the problem which consists of four blocks labelled A, B, C and D respectively.

Goal State: all blocks are in one stack ordered from A-Z, where A is in the bottom of the stack.

### Question:

Formulate the above problem as a search problem. Problem formulation means to describe the following: State Space, Initial State, Goal State, Actions, Transition model and Path Cost

### Solution:

#### State Representation:

- A state can be represented as a list of stacks of blocks. Each stack can be an ordered list of blocks, where the first element is the bottom block and the last element is the top block.
- For example, a state with four blocks (A, B, C, D) could be represented as [['A'], ['B'], ['C'], ['D']] if all blocks are on the table, or as [['A', 'B'], ['C', 'D']] if B is on top of A and C is on the table D on top of C.

#### Initial State:

- The initial configuration of the blocks.
- Example: [['A', 'B'], ['C', 'D']] means A is on the table, B is on top of A, and C is on the table, D on top of C.

#### Goal State:

- The desired configuration of the blocks.
- Example: [['A', 'B', 'C', 'D']] means all blocks A, B, C and D are in the same stack, A on the table, B on top of A, C on top of B and D on top of C.

#### Actions:

- The set of actions that can be performed to move from one state to another. In the block world, the actions typically include:
  - Move a block from the top of one stack to the top of another stack.
  - Move a block from the top of one stack to the table.
  - Move a block from the table to the top of another stack.
- Actions should only be applied when they result in valid states (i.e., a block can only be moved if it is at the top of a stack).

#### Transition Model:

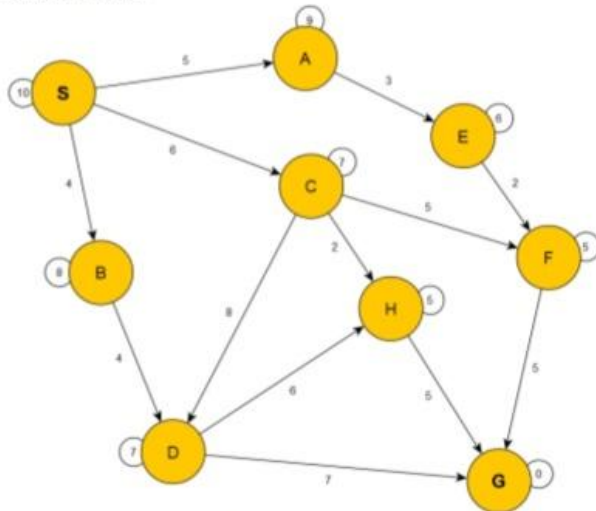
- The result of applying an action to a state. It defines how the state changes when an action is performed.
- Example: Moving block B from the stack [A, B] to the stack [C, D] results in the state  $[[ 'A' ], [ 'C', 'D', 'B' ]]$ .

#### Path Cost:

- The cost associated with moving from one state to another. Typically, each action can have a uniform cost, e.g., cost of 1 per move.

#### Question 3

Consider the following graph: The value inside the circle beside each node is the heuristic value of a heuristic function.



- 1- Is this heuristic admissible? Why?
- 2- Is this heuristic consistent? Why?
- 3- Perform the UCS algorithm; explicitly write down the queue at each step as well as the list of expanded nodes. Ties, when applicable, are expanded by alphabetical order.
- 4- Perform the A\* algorithm; explicitly write down the queue at each step as well as the list of expanded nodes. Ties, when applicable, are expanded by alphabetical order.
- 5- Perform the greedy algorithm; explicitly write down the queue at each step as well as the list of expanded nodes.

Solution:

- 1) Admissible: yes, since all the heuristic values are less than the real path cost.

| Node | $h(n)$ | Path Cost |
|------|--------|-----------|
| S    | 10     | 13        |
| A    | 9      | 10        |
| B    | 8      | 11        |
| C    | 7      | 7         |
| D    | 7      | 7         |
| E    | 6      | 7         |



|   |   |   |
|---|---|---|
| F | 5 | 5 |
| H | 5 | 5 |

2) Yes, since no node violates the rule :  $h(n) \leq PC(n, n') + h(n')$

3) See below table (UCS):

|                                  | Queue                                                                                | Expanded Nodes         |
|----------------------------------|--------------------------------------------------------------------------------------|------------------------|
|                                  | (S, 0)                                                                               |                        |
|                                  | (S, B, 4) (S, A, 5) (S, C, 6)                                                        | S                      |
|                                  | (S, A, 5) (S, C, 6) (S, B, D, 8)                                                     | S, B                   |
|                                  | (S, C, 6) (S, B, D, 8) (S, A, E, 8)                                                  | S, B, A                |
|                                  | (S, B, D, 8) (S, A, E, 8) (S, C, H, 8) <del>(S, C, D, 14)</del> (S, C, F, 11)        | S, B, A, C             |
|                                  | (S, A, E, 8) (S, C, H, 8) (S, C, F, 11) <del>(S, B, D, H, 14)</del> (S, B, D, G, 15) | S, B, A, C, D          |
| The solution is (S, C, H, G, 13) | (S, C, H, 8) (S, A, E, F, 10) <del>(S, C, F, 11)</del> (S, B, D, G, 15)              | S, B, A, C, D, E       |
|                                  | (S, A, E, F, 10) <del>(S, B, D, G, 15)</del> (S, C, H, G, 13)                        | S, B, A, C, D, E, H    |
|                                  | (S, C, H, G, 13) <del>(S, A, E, F, G, 15)</del>                                      | S, B, A, C, D, E, H, F |

| Queue                                                                         | Expanded Nodes |
|-------------------------------------------------------------------------------|----------------|
| (S, 10)                                                                       |                |
| (S, B, 12) (S, C, 13) (S, A, 14)                                              | S              |
| (S, C, 13) (S, A, 14) (S, B, D, 15)                                           | S, B           |
| (S, C, H, 13) (S, A, 14) (S, B, D, 15) (S, C, F, 16) <del>(S, C, D, 21)</del> | S, B, C        |
| (S, C, H, G, 13) (S, A, 14) (S, B, D, 15) (S, C, F, 16)                       | S, B, C, H     |
|                                                                               | S, B, C, H, G  |

4) See below table (A\*):

The solution is (S, C, H, G, 13)

5) See below table (Greedy):

| Queue                                  | Expanded Nodes |
|----------------------------------------|----------------|
| (S, 10)                                |                |
| (S, C, 7) (S, B, 8) (S, A, 9)          | S              |
|                                        |                |
| (S, C, F, 5) (S, C, H, 5) (S, C, D, 7) | S, C           |
| (S, C, F, G, 0)                        | S, C, F        |
|                                        | S, C, F, G     |

The solution is (S, C, F, G, 16)

### Ai Question 03:

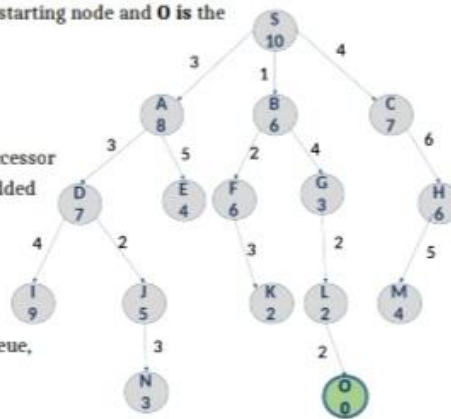
Consider the following search tree where **S** is the starting node and **O** is the goal node.

- **Costs are on edges**
- **Heuristic values on nodes**

Perform the following search strategies where successor function is in Left to Right order (left nodes added before the right nodes to the fringe):

For each algorithm you have to:

- Precise the type of the used fringe (queue, stack, cost priority, heuristic priority, ...)
- Give the content of the fringe at each step
- List the explored nodes
- State the found path and its cost



#### A. [15 pts] BFS - Breadth First

| Step | Fringe type is Queue (FIFO)    | Explored nodes                                 |
|------|--------------------------------|------------------------------------------------|
| 1    | S                              |                                                |
| 2    | SA, SB, SC                     | S                                              |
| 3    | SB, SC, SAD, SAE               | S, A                                           |
| 4    | SC, SAD, SAE, ABF, SBG         | S, A, B                                        |
| 5    | SAD, SAE, ABF, SBG, SCH        | S, A, B, C                                     |
| 6    | SAE, ABF, SBG, SCH, ASDI, SADJ | S, A, B, C, D                                  |
| 7    | ABF, SBG, SCH, ASDI, SADJ      | S, A, B, C, D, E                               |
| 8    | SBG, SCH, ASDI, SADJ, ABFK     | S, A, B, C, D, E, F                            |
| 9    | SCH, ASDI, SADJ, ABFK, SBGL    | S, A, B, C, D, E, F, G                         |
| 10   | ASDI, SADJ, ABFK, SBGL, SCHM   | S, A, B, C, D, E, F, G, H                      |
| 11   | SADJ, ABFK, SBGL, SCHM         | S, A, B, C, D, E, F, G, H, I                   |
| 12   | ABFK, SBGL, SCHM, SADJN        | S, A, B, C, D, E, F, G, H, I, J                |
| 13   | SBGL, SCHM, SADJN              | S, A, B, C, D, E, F, G, H, I, J, K             |
| 14   | SCHM, SADJN, SBGLO             | S, A, B, C, D, E, F, G, H, I, J, K, L          |
| 15   | SADJN, SBGLO                   | S, A, B, C, D, E, F, G, H, I, J, K, L, M       |
| 16   | SBGLO                          | S, A, B, C, D, E, F, G, H, I, J, K, L, M, N    |
| 17   |                                | S, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O |

Path: S → B → G → L → O

Cost: 1 + 4 + 2 + 2 = 9

**B. [15 pts] DFS - Depth First**

| Step | Fringe type is Stack (LIFO) | Explored nodes         |
|------|-----------------------------|------------------------|
| 1    | S                           |                        |
| 2    | SA, SB, SC                  | S                      |
| 3    | SA, SB, SCH                 | S, C                   |
| 4    | SA, SB, SCHM                | S, C, H                |
| 5    | SA, SB                      | S, C, H, M             |
| 6    | SA, SBF, SBG                | S, C, H, M, B          |
| 7    | SA, SBF, SBGL               | S, C, H, M, B, G       |
| 8    | SA, SBF, SBGLO              | S, C, H, M, B, G, L    |
| 9    |                             | S, C, H, M, B, G, L, O |
| 10   |                             |                        |

Path : S à B à G à L à O

Cost:  $1 + 4 + 2 + 2 = 9$

**C. [15 pts] UC – Uniform Cost**

| Step | Fringe type is Priority Queue (lowest path cost g(n)) | Explored nodes               |
|------|-------------------------------------------------------|------------------------------|
| 1    | (0)S                                                  |                              |
| 2    | (1)SB, (3)SA, (4)SC                                   | S                            |
| 3    | (3)SA, (3)SBF, (4)SC, (5)SBG                          | S, B                         |
| 4    | (3)SBF, (4)SC, (5)SBG, (6)SAD, (8)SAE                 | S, B, A                      |
| 5    | (4)SC, (5)SBG, (6)SAD, (8)SAE, (6)SBFK                | S, B, A, F                   |
| 6    | (5)SBG, (6)SAD, (8)SAE, (6)SBFK, (10)SCH              | S, B, A, F, C                |
| 7    | (6)SAD, (7)SBGL, (8)SAE, (6)SBFK, (10)SCH             | S, B, A, F, C, G             |
| 8    | (7)SBGL, (8)SAE, (6)SBFK, (8)SADJ, (10)SCH, (10)SADI  | S, B, A, F, C, G, D          |
| 9    | (8)SAE, (9)SBGLO, (6)SBFK, (8)SADJ, (10)SCH, (10)SADI | S, B, A, F, C, G, D, L       |
| 10   | (9)SBGLO, (6)SBFK, (8)SADJ, (10)SCH, (10)SADI         | S, B, A, F, C, G, D, L, E    |
| 11   |                                                       | S, B, A, F, C, G, D, L, E, G |

Path : S à B à G à L à O

Cost:  $1 + 4 + 2 + 2 = 9$



**D. [15 pts] Greedy**

| Step | Fringe type is Priority Queue (heuristic $h(n)$ ) | Explored nodes |
|------|---------------------------------------------------|----------------|
| 1    | 10S                                               |                |
| 2    | 6SB, 7SC, 8SA                                     | S              |
| 3    | 3SBG, 6SBF, 7SC, 8SA                              | S, B           |
| 4    | 2SBGL, 6SBF, 7SC, 8SA                             | S, B, G        |
| 5    | 0SBGLO, 6SBF, 7SC, 8SA                            | S, B, G, L     |
| 6    |                                                   | S, B, G, L, O  |
| 7    |                                                   |                |
| 8    |                                                   |                |
| 9    |                                                   |                |
| 10   |                                                   |                |

Path : S à B à G à L à O

Cost:  $1 + 4 + 2 + 2 = 9$

**E. [15 pts] A\***

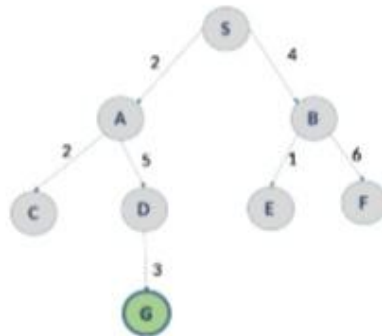
| Step | Fringe type is Priority Queue ( $f(n) = g(n) + h(n)$ ) | Explored nodes      |
|------|--------------------------------------------------------|---------------------|
| 1    | 10S                                                    |                     |
| 2    | 7SB, 11SA, 11SC                                        | S                   |
| 3    | 8SBG, 9SBF, 11SA, 11SC                                 | S, B                |
| 4    | 9SBF, 9SBGL, 11SA, 11SC                                | S, B, G             |
| 5    | 8SBFK, 9SBGL, 11SA, 11SC                               | S, B, G, F          |
| 6    | 9SBGL, 11SA, 11SC                                      | S, B, G, F, K       |
| 7    | 9SBGLO, 11SA, 11SC                                     | S, B, G, F, K, L    |
| 8    |                                                        | S, B, G, F, K, L, O |
| 9    |                                                        |                     |
| 10   |                                                        |                     |

Path : S à B à G à L à O

Cost :  $1 + 4 + 2 + 2 = 9$

**Problem 2: (25 points)**

Consider the following search tree where S is the starting node and G is the goal node, costs are on edges:



Is the following heuristic admissible and why?

| Node            | A | B | C | D | E | F | G |
|-----------------|---|---|---|---|---|---|---|
| Heuristic Value | 5 | 2 | 6 | 2 | 1 | 5 | 0 |

**Solution:**

- Find the **actual shortest cost** from each node to goal G

- $G \rightarrow G = 0$
- $D \rightarrow G = 3$
- $A \rightarrow D \rightarrow G = 5 + 3 = 8$
- $S \rightarrow A \rightarrow D \rightarrow G = 2 + 5 + 3 = 10$
- Other nodes (no path to G): B, C, E, F  $\rightarrow \infty$

- Compare  $h(n)$  with real cost

| Node | $h(n)$ | Real Cost | OK? |
|------|--------|-----------|-----|
| G    | 0      | 0         | Ok  |
| D    | 2      | 3         | OK  |
| A    | 5      | 8         | OK  |
| S    | 6      | 10        | OK  |

For all nodes:  $h(n) \leq$  actual cost. Thus, the heuristic is admissible.

## Ai Question 04:

### **Question 1:**

Convert the following sentences into FOL:

- 1- Each petition must have at least one report

$$\forall p(\text{Petition}(p) \rightarrow \exists r(\text{Report}(r) \wedge \text{Contains}(p, r)))$$

- 2- Each petition is owned by a student

$$\forall p(\text{Petition}(p) \rightarrow \exists s(\text{Student}(s) \wedge \text{Owns}(s, p)))$$

- 3- Each student must register at least one course

$$\forall s(\text{Student}(s) \rightarrow \exists c(\text{Course}(c) \wedge \text{Registers}(s, c)))$$

- 4- Intelligent student gets grade A in all registered courses

$$\forall s \forall c(\text{Student}(s) \wedge \text{Course}(c) \wedge \text{Registers}(s, c) \wedge \text{Intelligent}(s) \rightarrow \text{Grade}(s, c, A))$$

- 5- Active student solves all assignments

$$\forall s \forall a(\text{Student}(s) \wedge \text{Assignment}(a) \wedge \text{Active}(s) \rightarrow \text{Solves}(s, a))$$

- 6- Lazy students are sleepy

$$\forall x(\text{Student}(x) \wedge \text{Lazy}(x) \rightarrow \text{Sleepy}(x))$$

- 7- Polite students listen to the instructor

$$\forall x(\text{Student}(x) \wedge \text{Polite}(x) \rightarrow \text{ListensToInstructor}(x))$$

- 8- One absent allowed per student

$$\forall x(\text{Student}(x) \rightarrow \text{AllowedAbsence}(x, 1))$$

- 9- Un-polite students makes a lot of noise

$$\forall x(\text{Student}(x) \wedge \text{UnPolite}(x) \rightarrow \text{MakesNoise}(x))$$

- 10- Un-polite students makes a lot of noise during all exams

$$\forall x \forall y(\text{Student}(x) \wedge \text{UnPolite}(x) \wedge \text{Exam}(y) \rightarrow \text{MakesNoise}(x, y))$$

## Question 2:

Convert the following sentences into FOL:

- 1- Each desk must have at least one computer

$$\forall x(\text{Desk}(x) \rightarrow \exists y(\text{Computer}(y) \wedge \text{Has}(x,y)))$$

- 2- Each computer is owned by an employee

$$\forall x(\text{Computer}(x) \rightarrow \exists y(\text{Employee}(y) \wedge \text{OwnedBy}(x,y)))$$

- 3- Each employee must have at least one desk

$$\forall x(\text{Employee}(x) \rightarrow \exists y(\text{Desk}(y) \wedge \text{Has}(x,y)))$$

- 4- Active employee completes all assigned tasks

$$\forall x \forall y((\text{Employee}(x) \wedge \text{Active}(x) \wedge \text{Task}(y) \wedge \text{Assigned}(x,y)) \rightarrow \text{Completes}(x,y))$$

- 5- Practical employee respects all other employees

$$\forall x \forall y((\text{Employee}(x) \wedge \text{Practical}(x) \wedge \text{Employee}(y) \wedge (x \neq y)) \rightarrow \text{Respects}(x,y))$$

- 6- Lazy employees are sleepy

$$\forall x(\text{Employee}(x) \wedge \text{Lazy}(x) \rightarrow \text{Sleepy}(x))$$

- 7- Polite employees listen to the manager

$$\forall x(\text{Employee}(x) \wedge \text{Polite}(x) \rightarrow \text{ListensToManager}(x))$$

- 8- Each employee has one manager

$$\forall x (\text{Employee}(x) \rightarrow \exists y (\text{Manager}(y) \wedge \text{Manages}(y,x)))$$

- 9- Unpolite employees do not obey his manager.

$$\forall x \forall y(\text{Employee}(x) \wedge \text{Unpolite}(x) \wedge \text{Manager}(y,x) \rightarrow \neg \text{Obeys}(x,y))$$

- 10- Un-polite employee bothers all other employees

$$\forall x(\text{Employee}(x) \wedge \text{UnPolite}(x) \rightarrow \forall y(\text{Employee}(y) \rightarrow (y \neq x \rightarrow \text{Bothers}(x,y))))$$

### **Question 3:**

Consider the following sentences in FOL:

1.  $F_1 = \forall x \forall y (C(y, x) \rightarrow F(y)) \rightarrow H(x)$
2.  $F_2 = \forall x (G(x) \rightarrow F(x))$
3.  $F_3 = \forall x \exists y (C(x, y) \wedge G(y)) \rightarrow G(x)$
4.  $F_4 = \forall x (G(x) \rightarrow H(x))$

where:

|                |                                           |
|----------------|-------------------------------------------|
| <b>C(x,y):</b> | x is a dragon that is a child of dragon y |
| <b>F(x):</b>   | x is a dragon that can fly                |
| <b>H(x):</b>   | x is a dragon that is happy               |
| <b>G(x) :</b>  | x is a dragon that is green               |

**1- Translate each of the above sentences into plain English.**

$$F_1 = \forall x \forall y (C(y, x) \rightarrow F(y)) \rightarrow H(x)$$

Every dragon is happy if all its children can fly

$$F_2 = \forall x (G(x) \rightarrow F(x))$$

Green dragons can fly.

$$F_3 = \forall x \exists y (C(x, y) \wedge G(y)) \rightarrow G(x)$$

A dragon is green if it is a child of at least one green dragon

$$F_4 = \forall x (G(x) \rightarrow H(x))$$

All green dragons are happy

## Ai Question 05:

### Exercise 1:

Express the following sentences in FOL using the below vocabulary:

- Unary predicates: Instructor, Student, and PC.
- Binary Predicates: Study, Passed, Hates, Loves, Friend and Owns.
- Tertiary Predicate: Takes (x, y, z) means student x is taking course y with instructor z.

1. Every student who studying AI course has passed the algorithm course

$$\forall x \text{ Student}(x) \wedge \text{Study}(x, \text{AI}) \Rightarrow \text{Passed}(x, \text{Algorithm})$$

2. Every student who studying the javaLab course he is studying the java course too

$$\forall x \text{ Student}(x) \wedge \text{Study}(x, \text{JavaLab}) \Rightarrow \text{Study}(x, \text{Java})$$

3. The student Sami is taking at least 2 courses with the instructor Rami

$$\exists x, y \text{ Takes}(\text{Sami}, x, \text{Rami}) \wedge \text{Takes}(\text{Sami}, y, \text{Rami}) \wedge (x \neq y)$$

4. There is an instructor such that the student Sami is taking 2 courses with him

$$\exists x \exists y \exists z \text{ Instructor}(z) \wedge \text{Takes}(\text{Sami}, x, z) \wedge \text{Takes}(\text{Sami}, y, z) \wedge (x \neq y)$$

5. A student can't take more than 2 courses with the same instructor

$$\forall x \forall y \forall s \forall t ((\text{Student}(x) \wedge \text{Instructor}(y) \wedge \text{Takes}(x, s, y) \wedge \text{Takes}(x, t, y) \wedge s \neq t) \rightarrow \\ \neg \exists z (\text{Takes}(x, z, y) \wedge z \neq s \wedge z \neq t))$$

6. All java students have a PC

$$\forall x \text{ Student}(x) \wedge \text{Study}(x, \text{Java}) \Rightarrow \exists y \text{ PC}(y) \wedge \text{Owns}(x, y)$$

7. There is an instructor who is loved by every student



$$\exists x \forall y \text{ Instructor}(x) \wedge \text{Student}(y) \wedge \text{loves}(y, x)$$

8. If 2 students are taking the same course with the same instructor then they are friends

$$\forall x, y, z, s, t \text{ Student}(x) \wedge \text{Student}(y) \wedge \text{Instructor}(z) \wedge \text{Takes}(x, s, z) \wedge \text{Takes}(y, t, z) \wedge (s = t) \Rightarrow \text{Friend}(x, y)$$

9. No one can own all things he loves

$$\neg \exists x \forall y (\text{Loves}(x, y) \rightarrow \text{Owns}(x, y))$$

or equivalently:

$$\forall x \exists y (\text{Loves}(x, y) \wedge \neg \text{Owns}(x, y)) \quad \forall x \exists y (\text{Loves}(x, y) \wedge \neg \text{Owns}(x, y))$$

10. A student taking a course with an instructor and he hates this instructor will not pass this course

$$\forall x \forall y \forall z \text{ Student}(x) \wedge \text{Instructor}(y) \wedge \text{Takes}(x, z, y) \wedge \text{Hates}(x, y) \Rightarrow \neg \text{Passed}(x, z)$$

### Exercise 2:

a) Find the substitution, if exists, that unifies Saw(x, Bell) with:

- 1- Saw(Patrick, Bell)
- 2-  $\forall x \text{ Saw}(\text{Howard}, x)$
- 3-  $\forall y \text{ Saw}(y, y)$
- 4-  $\forall z \text{ Saw}(\text{Bell}, z)$
- 5-  $\forall x \text{ Saw}(\text{Spouse}(x), x)$

1. UNIFY(Saw(x, Bell), Saw(Patrick, Bell)) = {x/ Patrick }
2. UNIFY(Saw(x, Bell), Saw (Howard, x)) = fail,  
Standardising apart: UNIFY(Saw(x, Bell), Saw (Howard, x7)) = {x/ Howard ,x7= Bell }
3. UNIFY(Saw(x, Bell), Saw(y, y)) = {y/Bell , x/Bell }
4. UNIFY(Saw(x, Bell), Saw(Bell, z) ) = {x/Bell , z/Bell }
5. UNIFY(Saw(x, Bell), Saw(Spouse(x), x)) = fail  
Standardising apart:  
UNIFY(Saw(x1, Bell), Saw(Spouse(x), x)) = { x1/ Spouse(Bell), x/Bell }

b) Write a small paragraph in English that describes what was part a about.

The question in part a was about a query - Saw(x, Bell) – “who saw Bell?”  
Given the following facts:

- 1- Saw(Patrick, Bell) – Patrick saw Bell
- 2-  $\forall x \text{ Saw}(\text{Howard}, x)$  – Howard saw everyone
- 3-  $\forall y \text{ Saw}(y, y)$  – Everyone saw him/herself
- 4-  $\forall z \text{ Saw}(\text{Bell}, z)$  – Bell saw everyone
- 5-  $\forall x \text{ Saw}(\text{Spouse}(x), x)$  – Everyone saw his/her spouse

‘Logic’ and substitutions will infer the following answers:

Patrick saw Bell  
Howard saw bell  
Bell saw himself  
Bell’s spouse saw Bell

AI Question 06:

**Question 1:**

Convert the following FOL sentences into CNF:

Note 1: There is no need to standardize/name variables apart.

Note 2: You should specify the name and details of each step.

1-  $\forall x [\forall y C(y) \Rightarrow P(x, y)] \Rightarrow [\exists z P(z, x)]$

Eliminate implication

$$\forall x [\forall y \neg C(y) \vee P(x, y)] \Rightarrow [\exists z P(z, x)]$$

$$\forall x [\neg \forall y \neg C(y) \vee P(x, y)] \vee [\exists z P(z, x)]$$

Drive in negation

$$\forall x [\exists y C(y) \wedge \neg P(x, y)] \vee [\exists z P(z, x)]$$

Skolemize

$$\forall x [C(F(x)) \wedge \neg P(x, F(x))] \vee P(G(x), x)$$

Drop universal quantifiers

$$[C(F(x)) \wedge \neg P(x, F(x))] \vee P(G(x), x)$$

Distribute  $\wedge$  over  $\vee$

$$[C(F(x)) \vee P(G(x), x)] \wedge [\neg P(x, F(x)) \vee P(G(x), x)]$$

2-  $\exists x \forall y H(x, y)$

Skolemize

$$\forall y H(\text{Peter}, y)$$

Drop universal quantifiers

$$H(\text{Peter}, y)$$

3-  $\forall y \exists x L(x, y)$

Skolemize

$$\forall y L(F(y), y)$$

Drop universal quantifiers

$$L(F(y), y)$$

$$4- \forall z A(z) \Rightarrow [\forall t B(t) \Rightarrow \neg C(z, t)]$$

Eliminate implication

$$\forall z \neg A(z) \vee [\forall t B(t) \Rightarrow \neg C(z, t)]$$

$$\forall z \neg A(z) \vee [\forall t \neg B(t) \vee \neg C(z, t)]$$

Drop universal quantifiers

$$\neg A(z) \vee \neg B(t) \vee \neg C(z, t)$$

$$5- \forall w [\exists x \text{Pop}(x) \wedge \text{Lu}(w, x)] \Rightarrow S(w)$$

Eliminate implication

$$\forall w [\neg \exists x \text{Pop}(x) \wedge \text{Lu}(w, x)] \vee S(w)$$

Drive in negation

$$\forall w \forall x [\neg (\text{Pop}(x) \wedge \text{Lu}(w, x))] \vee S(w)$$

$$\forall w \forall x \neg \text{Pop}(x) \vee \neg \text{Lu}(w, x) \vee S(w)$$

Drop universal quantifiers

$$\neg \text{Pop}(x) \vee \neg \text{Lu}(w, x) \vee S(w)$$

$$6- [\text{Bcc}(G(x)) \wedge \neg \text{Cc}(x, G(x))] \vee \text{Cc}(H(x), x)$$

Distribute  $\wedge$  over  $\vee$

$$[\text{Bcc}(G(x)) \vee \text{Cc}(H(x), x)] \wedge [\neg \text{Cc}(x, G(x)) \vee \text{Cc}(H(x), x)]$$

### Ai Question 07:

Consider the following relations definition, facts and rules in prolog:

**Relations definition:**

- fplayer(X): X is a football player
- mplayer(X): X is a music player
- plays(P,T): P plays in the team T
- friend(X,Y): X is the friend of Y
- sportive (X): X is sportive

**Rules:**

- friend(X,Y):-friend(Y,X).
- sportive(X):-fplayer(X).
- friend(X,Y):-plays(X,Z),plays(Y,Z).
- samehobby(X,Y):-fplayer(X),fplayer(Y).

**Facts:**

- fplayer (samir).
- fplayer(amir).
- fplayer(hadi)
- mplayer(nada).
- mplayer(hadi).
- mplayer(rami).
- sportive(jad).
- plays(samir,tigers).
- plays(hadi,tigers).
- plays(amir,tigers).
- plays(rami,dream)
- plays(nada,birds).
- plays(hadi,birds).
- friend(hadi,adel).

friend(X, hadi)

- friend(X, Y):- friend(Y, X).
  - o friend(hadi, X)

- X = adel
  - friend(X,Y):-plays(hadi,Z),plays(Y,Z).
    - o Plays(hadi, Z)
      - Z = tigers
        - Plays(Y, tigers)
          - Y = samir
          - Y = hadi
          - Y = amir
      - Z = birds
        - Plays(Y, birds)
          - Y = nada
          - Y = hadi

#### 1- Convert the three rules to English

- friend(X,Y):-friend(Y,X).  
**Friendship is mutual**
- sportive(X):-fplayer(X).  
**All football players are sportive**
- sportive(X):-friend(hadi,X).  
**All hadi friends are sportive**
- friend(X,Y):-plays(X,Z),plays(Y,Z).  
**Persons that plays for the same team are friends**

#### 2- Write the following questions in prolog:

- Who plays both music and football?  
**?- mplayer(X), fplayer(X).**

- Who are the players of the team tigers?  
?- plays(X, tigers).
- In which teams play booth ahmad and jad?  
?- plays(ahmad, X), plays(jad, X).
- Is ahmad a football player ?  
?- fplayer(ahmad).
- Is jad a music player and plays in the chicken team?  
?- mplayer(jad), plays(jad, chicken).
- Who are friends but at least one of them is sportive?  
?- friend(X, Y), sportive(X); friend(X, Y), sportive(Y).

### 3- What would be the answer to each of the following questions:

- ?- friend(X, hadi).  
X=adel  
X=samir  
X=hadi  
X=amir  
X=nada  
X=hadi
- ?-friend(hadi, X), sportive(X).  
X=samir  
X=hadi  
X=amir



- ?-plays(hadi,X),plays(hadi,Y).  
**X=tigers**  
**Y=tigers;**  
**X=tigers**  
**Y=birds;**  
**X=birds**  
**Y=tigers;**  
**X=birds**  
**Y=birds;**

3- What would be the answer to each of the following questions:

- ?-plays(hadi,X),plays(hadi,Y), not(X=Y).  
**X=tigers**  
**Y=birds;**  
**X=birds**  
**Y=tigers;**
- ?-plays(hadi,X),plays(nada,X).  
**X=birds**
- ?-plays(hadi,X),plays(adel,X).  
**fail**
- ?-fplayer(X),mplayer(X).  
**X=hadi**
- ?-fplayer(X),mplayer(Y).  
**X=samir**  
**Y=nada;**  
**X=samir**  
**Y=hadi;**  
**X=samir**  
**Y=rami;**

```
X=amir  
Y=nada;  
X=amir  
Y=hadi;  
X=amir  
Y=rami;  
X=hadi  
Y=nada;  
X=hadi  
Y=hadi;  
X=hadi  
Y=rami;
```

- ?-fplayer(samir),friend(samir,hadi),sportive(amir).

```
true
```